

PROJET DE STAGE — FILIÈRE INFORMATIQUE

Plan de révision en 5 jours

Portail Carrière — Application web de recrutement
React · Node.js / Express · Supabase

Volume total du projet : 4 519 lignes

• Backend : 1 296 lignes | Frontend : 3 223 lignes

Couvert par ce plan : 3 273 lignes

soit 31 fichiers sur 43

Rythme conseillé : 4 à 5 h par jour

Vue d'ensemble

Jour	Thème	Volume	Charge de travail
J1	Architecture & configuration	~250 lignes	Légère (déjà entamée)
J2	Flux complet des Offres	~700 lignes	La plus grosse journée
J3	Candidatures & upload de fichiers	~800 lignes	La plus technique
J4	Authentification & sécurité	~400 lignes	Moyenne
J5	Révision & simulation de soutenance	—	Aucun code nouveau

Trois règles pour tenir le rythme

1. **Ne lisez jamais un fichier isolément.** Suivez toujours un fil complet : URL → route → contrôleur → Supabase → retour React. C'est exactement ce que le jury vous demandera de faire.
2. **Ignorez complètement** `styles/index.css` (886 lignes de décoration) et `scripts/seed0ffres.js` (données de démonstration). Aucun jury ne vous interrogera dessus.
3. **Faites la manipulation proposée après chaque module.** Voir le code s'exécuter ancre bien mieux la compréhension que le relire une deuxième fois.

Si vous prenez du retard

Sacrifiez dans cet ordre : `FormulaireOffre.jsx` (283 l., CRUD classique), puis `CandidaturesOffre.jsx`, puis `seedOffres.js`.

Ne sacrifiez jamais `candidatures.controller.js` ni `auth.middleware.js` : ce sont les deux fichiers sur lesquels un jury creuse systématiquement.

Déjà couverts avant le démarrage de ce plan : `server.js`, `app.js`, `supabaseClient.js` et `offres.routes.js` — soit l'essentiel du Jour 1.

Architecture & configuration

Objectif : être capable de dessiner l'architecture au tableau et de justifier chaque choix technique.

Fichiers à étudier

Fichier	Lignes	État / durée
backend/src/server.js	14	fait
backend/src/app.js	43	fait
backend/src/config/supabaseClient.js	29	fait
backend/src/config/constantes.js	26	10 min
backend/.env + .env.example + .gitignore	—	15 min
frontend/src/main.jsx	15	10 min
frontend/src/App.jsx	61	45 min ★
frontend/src/services/axiosClient.js	59	45 min ★
frontend/index.html + vite.config.js	—	10 min

Concepts théoriques à maîtriser

- **L'architecture 3-tiers** : Présentation (React) → Métier (Express) → Données (Supabase)
- **La règle d'or du projet** : le frontend ne parle **jamais** directement à Supabase
- **CORS** : pourquoi le port 5173 ne peut pas appeler le port 5000 sans autorisation explicite
- **Variables d'environnement** : pourquoi `.env` n'est jamais commité, rôle du `.env.example`
- **anon key vs service_role key** : la différence de pouvoir, et pourquoi la seconde reste côté serveur
- **React Router** : BrowserRouter, Routes, Route, routes imbriquées, `<Outlet />`
- **Les intercepteurs Axios** : ajouter le token automatiquement, réagir à une réponse 401

Questions probables du jury

Question	Ce qu'il faut répondre
« Pourquoi un backend Express, alors que Supabase expose déjà une API REST ? » ★★	<i>La question centrale de ce projet.</i> Pour garder la clé service_role côté serveur, centraliser la validation des données, et implémenter la logique métier (anti-doublon, contrôle d'expiration, URL signées). Sans backend, il faudrait exposer la clé et écrire des politiques RLS complexes.
« Pourquoi séparer app.js et server.js ? »	app.js configure, server.js démarre. Permet de tester l'application sans ouvrir de port réseau.
« Que se passe-t-il si votre .env part sur GitHub ? »	Accès total à la base pour n'importe qui : lecture, modification, suppression, y compris les CV. D'où le .gitignore.
« Qu'est-ce qu'un middleware ? »	Une fonction exécutée entre la requête et la réponse, dans une chaîne ordonnée.
« Pourquoi 3 dossiers backend : routes, controllers, config ? »	Séparation des responsabilités, recommandée en section 7.5 du cahier des charges.

Exercice du jour

Dessinez **de mémoire, sur papier**, le schéma des 3 couches avec les ports (5173, 5000, Supabase cloud) et les technologies de communication (Axios/HTTP, SDK Supabase). Vous devrez savoir le refaire au tableau.

Le flux complet des Offres

Objectif : maîtriser un aller-retour complet, de la saisie React jusqu'à PostgreSQL. C'est le fil rouge le plus simple et le plus interrogé.

Fichiers à étudier — dans cet ordre exact

#	Fichier	Lignes	Priorité
1	backend/src/routes/offres.routes.js	30	fait
2	backend/src/controllers/offres.controller.js	350	★★★ le cœur
3	frontend/src/services/offres.js	78	★★
4	frontend/src/pages/ListeOffres.jsx	143	★★★
5	frontend/src/components/OffreCard.jsx	32	★
6	frontend/src/pages/DetailOffre.jsx	137	★★
7	frontend/src/pages/DashboardRH.jsx	239	★★
8	frontend/src/pages/FormulaireOffre.jsx	283	★★

Conseil de découpage

Le contrôleur est séparé en deux blocs par des commentaires. Étudiez le matin les **3 fonctions publiques** (lignes 29 à 117), et l'après-midi les **7 fonctions RH** (lignes 129 à 350).

Concepts théoriques à maîtriser

Backend

- `async / await` et la structure `try / catch / next(err)`
- Le duo `{ data, error }` retourné systématiquement par le SDK Supabase
- Le query builder : `.from().select().eq().or().order().range()`
- La pagination : `range(début, fin)` combiné à `count: 'exact'`
- Les codes HTTP : **200** (OK), **201** (créé), **400** (données invalides), **404** (introuvable)
- La jointure `Candidature(count)` pour compter les candidatures par offre

Frontend

- `useState` : l'état d'un composant, pourquoi on ne modifie jamais une variable directement
- `useEffect` : quand se déclenche-t-il ? Rôle du tableau de dépendances `[]` vs `[page]`
- Le **debounce** de 300 ms dans `ListeOffres.jsx` : éviter une requête à chaque lettre tapée
- La fonction de nettoyage `return () => clearTimeout(...)`
- `useParams()` pour lire le `:id` de l'URL
- Les formulaires contrôlés : `value` + `onChange`

Questions probables du jury

Question	Ce qu'il faut répondre
« Comment une offre expirée disparaît-elle du site ? » ★★	Un filtre SQL dans <code>listOffresPubliques</code> : <code>statut = 'Publiée'</code> ET (<code>date_expiration</code> est null OU $\geq$ aujourd'hui). Aucune tâche planifiée n'est nécessaire.
« Différence entre archiver et supprimer ? » ★★	Archiver = <code>UPDATE statut = 'Archivée'</code> (réversible, les données restent). Supprimer = <code>DELETE</code> définitif, qui efface aussi les candidatures et les CV du Storage .
« Pourquoi /filtres doit-il être déclaré avant /:id ? »	Express s'arrête à la première route correspondante ; <code>/:id</code> capturerait le mot « filtres » comme un identifiant.
« À quoi sert le tableau [] de <code>useEffect</code> ? » ★★	Il liste les valeurs surveillées. [] = exécution unique au montage. [page] = ré-exécution à chaque changement de page. Oublier le tableau provoque une boucle infinie .
« Pourquoi avoir paginé ? »	Exigence 4.3 du cahier des charges : ne pas charger des centaines de lignes en une seule requête.
« Comment comptez-vous les candidatures par offre ? »	Jointure Supabase <code>select('*', Candidature(count))</code> : une seule requête au lieu du problème N+1.
« Pourquoi un délai de 300 ms sur la recherche ? »	Sans lui, taper « développeur » enverrait 11 requêtes successives au serveur.

Exercice du jour

Ouvrez <http://localhost:5173/offres> , appuyez sur **F12** → **onglet Réseau**, puis tapez dans la barre de recherche. Observez la requête `GET /api/offres?motCle=...` partir, et sa réponse JSON revenir. **Faites correspondre chaque paramètre de l'URL avec une ligne précise du contrôleur.**

Candidatures & upload de fichiers

Objectif : maîtriser la fonctionnalité la plus complexe du projet. C'est ici que le jury vous testera vraiment.

Fichiers à étudier

#	Fichier	Lignes	Priorité
1	backend/src/middlewares/upload.middleware.js	34	★★★
2	backend/src/controllers/candidatures.controller.js	363	★★★ le plus dur
3	backend/src/middlewares/erreur.middleware.js	40	★★
4	frontend/src/services/candidatures.js	68	★★★★
5	frontend/src/pages/FormulaireCandidature.jsx	356	★★★★
6	frontend/src/pages/CandidaturesOffre.jsx	181	★★
7	frontend/src/pages/DetailCandidature.jsx	211	★★

Priorité absolue

Concentrez-vous sur la fonction `creerCandidature` (lignes 45 à 180 du contrôleur). Elle enchaîne 8 étapes numérotées en commentaires. **Sachez les réciter dans l'ordre** — c'est la question de soutenance la plus probable de tout le projet.

Concepts théoriques à maîtriser

- **multipart/form-data vs application/json** : pourquoi un fichier binaire ne peut pas transiter en JSON
- **FormData** côté React : construire une requête mêlant fichier et champs texte
- **Multer** : `memoryStorage()`, `limits`, `fileFilter`, et la distinction `req.file` / `req.body`
- **Supabase Storage** : `bucket` privé, `.upload()`, `.createSignedUrl()`, `.remove()`
- **L'URL signée temporaire** : pourquoi 5 minutes, et pourquoi pas un bucket public
- **Le « rollback manuel »** : supprimer le CV uploadé si l'insertion en base échoue ensuite
- Le code **409 Conflict** et la contrainte SQL `unique(id_offre, id_candidat)`
- **La double validation** : côté React (confort utilisateur) **et** côté serveur (sécurité réelle)

Le schéma à connaître par cœur

1. React : l'utilisateur remplit le formulaire + choisit un fichier
2. React : construction d'un `FormData` (champs texte + fichier)
3. Axios : `POST /api/offres/:id/candidatures` en `multipart/form-data`
4. Express : cors OK -> Multer extrait le fichier dans `req.file`
5. Contrôleur : validation -> offre valide ? -> anti-doublon -> upload Storage
-> création/réutilisation du Candidat -> création de la Candidature
6. Supabase : PostgreSQL enregistre les lignes, Storage enregistre le fichier
7. Express : réponse 201 Created { success: true }
8. React : navigation vers l'écran de confirmation

Questions probables du jury

Question	Ce qu'il faut répondre
« Décrivez le trajet complet d'un CV, du clic jusqu'au stockage. » ★★★	La question reine. Récitez le schéma en 8 étapes ci-dessus.
« Pourquoi memoryStorage et pas diskStorage ? » ★★	Le fichier n'a pas vocation à rester sur le serveur : il transite en mémoire (req.file.buffer) puis part directement vers Supabase Storage. Cela évite les fichiers orphelins sur le disque.
« Comment empêchez-vous une double candidature ? » ★★	Deux niveaux : (1) une contrainte SQL unique(id_offre, id_candidat) — la vraie garantie ; (2) un contrôle applicatif <i>avant</i> l'upload, pour éviter d'envoyer un fichier inutilement. L'erreur PostgreSQL 23505 est traduite en HTTP 409.
« Que se passe-t-il si le CV est uploadé mais que l'insertion en base échoue ? » ★★★	<i>Question piège de niveau avancé.</i> Le code supprime le fichier du Storage (.remove()) pour ne pas laisser de fichier orphelin. C'est un rollback manuel, car Supabase Storage n'est pas transactionnel avec PostgreSQL.
« Comment garantisiez-vous la confidentialité des CV ? » ★★	Bucket privé , aucune URL publique. Le backend génère une URL signée valable 5 minutes, uniquement pour le RH authentifié (section 4.5, données personnelles).
« Un candidat peut-il contourner la limite de 5 Mo ? »	Non : la validation React est un confort, mais Multer refuse le fichier côté serveur via limits.fileSize.
« Pourquoi les tables Candidat et Candidature ? »	Candidat = la personne (unique par email). Candidature = l'acte de postuler. Une personne peut postuler à plusieurs offres sans dupliquer ses coordonnées.
« Postuler avec JEAN@mail.com puis jean@mail.com, ça passe ? »	Non : l'email est normalisé en minuscules avant toute recherche en base.

Authentification & sécurité

Objectif : c'est le sujet préféré des jurys informatiques. Deux questions pièges classiques vous attendent.

Fichiers à étudier

#	Fichier	Lignes	Priorité
1	backend/src/scripts/seedRh.js	63	★★
2	backend/src/controllers/auth.controller.js	81	★★★★
3	backend/src/middlewares/auth.middleware.js	35	★★★★
4	backend/src/routes/auth.routes.js	22	★
5	backend/src/routes/rh.routes.js	51	★★★★
6	frontend/src/services/auth.js	34	★★
7	frontend/src/services/axiosClient.js (<i>intercepteurs</i>)	59	★★★★
8	frontend/src/components/RouteProtegee.jsx	28	★★★★
9	frontend/src/pages/ConnexionRH.jsx	101	★★
10	frontend/src/components/LayoutRH.jsx	66	★

Concepts théoriques à maîtriser

- **Hachage vs chiffrement** : bcrypt est **irréversible**, ce n'est pas du chiffrement
- Le **sel** (salt) de bcrypt et le coût 10 : pourquoi un hachage volontairement lent
- `bcrypt.compare()` : on ne déchiffre jamais, on re-hache et on compare
- **Structure d'un JWT** : header.payload.signature — et le fait qu'il soit **signé, pas chiffré**
- `jwt.sign()` / `jwt.verify()`, rôle du `JWT_SECRET`, expiration (8 h)
- L'en-tête `Authorization: Bearer <token>`
- `router.use(verifierAuth)` : protéger **toutes** les routes suivantes en une seule ligne
- L'intercepteur Axios : injection automatique du token, déconnexion sur 401
- Les codes **401** (non authentifié) vs **403** (authentifié mais non autorisé)

Questions probables du jury

Question	Ce qu'il faut répondre
« Un JWT, c'est chiffré ? » ★★★	NON — piège classique. Un JWT est signé , pas chiffré. N'importe qui peut décoder son contenu en base64 (démonstration possible sur jwt.io). La signature garantit seulement qu'il n'a pas été modifié . C'est pourquoi on n'y met jamais de mot de passe ni de donnée sensible — seulement un id, un email, un nom.
« Votre RouteProtegee React suffit-elle à sécuriser l'espace RH ? » ★★★	NON — deuxième piège classique. C'est un confort d'affichage : le code React s'exécute chez le client, qui peut le modifier. La vraie sécurité est le middleware <code>verifierAuth</code> côté backend , qui vérifie le token à chaque requête. Preuve : un appel curl sur <code>/api/rh/offres</code> sans token renvoie 401.
« Pourquoi bcrypt plutôt que MD5 ou SHA-256 ? » ★★	bcrypt intègre un sel unique (deux mots de passe identiques donnent deux hashes différents) et il est volontairement lent , ce qui rend les attaques par force brute impraticables. MD5 et SHA sont rapides — donc inadaptés aux mots de passe.
« Pourquoi le même message d'erreur si l'email est faux ou si le mot de passe est faux ? » ★	Pour empêcher l' énumération de comptes : un attaquant ne doit pas pouvoir deviner quels emails existent en base.
« Où stockez-vous le token, et quel est le risque ? »	Dans le <code>localStorage</code> . Avantage : simple, survit au rechargement. Risque : vulnérable au XSS . Une alternative plus sûre serait un cookie <code>httpOnly</code> . <i>Savoir le dire montre de la maturité technique.</i>
« Pourquoi aucune route <code>/register</code> ? »	Exigence F08bis : un compte RH unique, créé par seeding (<code>npm run seed</code>). Aucun formulaire d'inscription n'est exposé.
« Que se passe-t-il à l'expiration du token ? »	Le backend renvoie 401, l'intercepteur Axios vide le <code>localStorage</code> et redirige vers la page de connexion (exigence F09).

Exercice du jour — à refaire devant le jury si on vous le demande

1. Ouvrez F12 → Application → Local Storage : observez le token après connexion.
2. Copiez-le, collez-le sur jwt.io → vous lisez son contenu en clair. C'est la démonstration parfaite de « signé ≠ chiffré ».
3. Dans un terminal : `curl http://localhost:5000/api/rh/offres` → réponse **401**. La sécurité est bien côté serveur.

Révision & simulation de soutenance

Aucun code nouveau. Cette journée sert à consolider et à répéter à voix haute.

Matin — Les 3 schémas à savoir dessiner au tableau

1. **L'architecture 3-tiers** avec les ports et les technologies de communication
2. **Le modèle relationnel** : Compte_RH (1) —< Offre (1) —< Candidature >— (1) Candidat
3. **Le flux d'une candidature** en 8 étapes (celui du Jour 3)

Midi — Les 3 traçages de flux à réciter à voix haute

Entraînez-vous à raconter chacun **sans regarder le code**, en nommant les fichiers traversés.

Flux	Difficulté	Fichiers à citer
A. « J'affiche la liste des offres »	Facile	ListeOffres.jsx → services/offres.js → axiosClient → app.js → offres.routes.js → offres.controller.js → Supabase → retour
B. « Je me connecte en tant que RH »	Moyen	ConnexionRH.jsx → services/auth.js → auth.routes.js → auth.controller.js (bcrypt.compare + jwt.sign) → localStorage → RouteProtegee
C. « J'envoie une candidature avec CV »	Difficile	FormulaireCandidature.jsx (FormData) → offres.routes.js (upload.single) → Multer → candidatures.controller.js (8 étapes) → Storage + PostgreSQL

Après-midi — Préparer la démonstration live

Répétez ce parcours **3 fois, chronométré** (visez 5 à 6 minutes) :

1. Page d'accueil -> liste des offres
2. Recherche par mot-clé (montrez le filtrage en direct)
3. Détail d'une offre -> formulaire de candidature
4. Tentative avec un fichier .txt -> message d'erreur (montre la validation)
5. Envoi d'un vrai PDF -> écran de confirmation
6. Re-tentative avec le même email -> erreur de doublon (montre F07)
7. Connexion RH -> tableau de bord (le compteur a augmenté)
8. Ouverture de la candidature -> téléchargement du CV
9. Changement de statut en « Accepté »

Préparez un plan B

Ayez des captures d'écran de chaque étape. Si le Wi-Fi tombe ou que Supabase est lent le jour J, vous ne serez pas bloquée devant le jury.

Balayage final — les 10 fichiers restants (45 min)

Ces fichiers ne figurent pas dans les journées 1 à 4 car ils n'apportent aucun concept nouveau. Un simple survol suffit, mais il vous permettra d'affirmer au jury que vous connaissez **chaque fichier** du projet.

Fichier	Lignes	Ce qu'il fait
pages/Accueil.jsx	71	Bannière + les 3 offres les plus récentes. Même structure que ListeOffres, en plus simple.
components/Pagination.jsx	35	Boutons Précédent / Suivant, réutilisés par 2 pages.
pages/ConfirmationCandidature.jsx	32	Écran de succès. Reçoit le titre de l'offre via le state de React Router.
components/Entete.jsx	23	Barre de navigation publique.
routes/candidatures.routes.js	22	Route d'information uniquement. Savoir dire pourquoi : l'envoi d'une candidature est rattaché à l'offre (POST /api/offres/:id/candidatures).
components/StatutBadge.jsx	21	Pastille colorée. Bon exemple de composant réutilisable si le jury en demande un.
components/LayoutPublic.jsx	21	Gabarit en-tête / contenu / pied. Illustre <Outlet /> de React Router.
components/PiedDePage.jsx	18	Pied de page.
components/Message.jsx	18	Bandeau d'alerte (erreur / succès / info).
components/Chargement.jsx	6	Affiche « Chargement en cours... ».

Deux fichiers volontairement exclus

`styles/index.css` (886 lignes de mise en forme) et `scripts/seedOffres.js` (93 lignes de données de démonstration) restent hors périmètre : aucun concept technique n'y est en jeu.

Fin de journée — Assumer les limites du projet

Un jury pose presque toujours : « **Que manque-t-il à votre application ?** ». Répondre honnêtement est **valorisant** — c'est de la lucidité technique, pas un aveu de faiblesse.

- **Pas de tests automatisés** — les tests ont été réalisés manuellement (Thunder Client, navigateur)
- **Un seul compte RH**, pas de gestion de rôles (explicitement hors périmètre, annexe 8.2)
- **Pas de notification email** au candidat lors d'un changement de statut (piste d'évolution)
- **Token en localStorage** — un cookie httpOnly serait plus robuste face au XSS
- **CORS ouvert à tous** en développement — à restreindre au domaine réel en production
- **Pas de politique de purge** des candidatures anciennes (évoquée en section 4.5, RGPD)

Récapitulatif visuel

	J1	J2	J3	J4	J5
Thème	Architecture	Offres	Candidatures	Sécurité	Révision
Backend	config	offres.controller	candidatures.controller + Multer	JWT + bcrypt	—
Frontend	App.jsx, Axios	useState / useEffect	FormData	RouteProtegee	—
Question reine	<i>Pourquoi Express ?</i>	<i>Gestion de l'expiration ?</i>	<i>Le trajet d'un CV ?</i>	<i>JWT chiffré ?</i>	<i>Ce qui manque ?</i>

Les 5 réponses à connaître par cœur

1. **Pourquoi un backend ?** Pour protéger la clé service_role, centraliser la validation et porter la logique métier.
2. **Le trajet d'un CV ?** FormData → Multer (mémoire) → Storage privé → chemin enregistré en base.
3. **Un JWT est signé, pas chiffré.** Son contenu est lisible ; seule son intégrité est garantie.
4. **La sécurité est côté serveur.** RouteProtegee n'est qu'un confort d'affichage.
5. **Anti-doublon à deux niveaux :** contrainte SQL unique + contrôle applicatif, traduits en HTTP 409.

Rappel des identifiants de démonstration

Espace RH — **rh@entreprise.com** / **MonMotDePasseSolide123!**

Backend : `cd backend && npm run dev` • Frontend : `cd frontend && npm run dev`

Plan de révision généré pour le projet « Portail Carrière » — React, Node.js/Express et Supabase. Documentation complète des endpoints disponible dans le fichier README.md à la racine du projet.